



UNIVERSITATEA DIN BUCUREȘTI

**FACULTATEA DE MATEMATICĂ ȘI
INFORMATICĂ**



SPECIALIZAREA INFORMATICĂ

Lucrare de licență

ARHITECTURA PAN CA MODEL PENTRU APLICAȚII WEB FULL-STACK

Studiu de caz privind reducerea risipei alimentare

Absolvent: Eduard Emanuel Dinea

Coordonator științific: Conf. univ. dr. Marius Iulian Mihăilescu

București, iulie 2026

Rezumat

Această lucrare prezintă proiectarea și implementarea unei aplicații [web full-stack](#), denumită [Kookio](#), realizată ca studiu de caz pentru analiza arhitecturii [PAN](#) ([Prisma](#)–[Analog](#)–[Nx](#)) în contextul aplicațiilor [web](#) moderne. Aplicația acoperă, într-un flux integrat, gestiunea cămării, planificarea zilnică a meselor și derivarea automată a listei de cumpărături pentru utilizatori casnici, cu potențial de a contribui la reducerea risipei alimentare prin diminuarea achizițiilor redundante.

Motivația lucrării derivă din necesitatea unor arhitecturi [web](#) care să asigure coerență între [frontend](#) și [backend](#), precum și un proces de dezvoltare unitar. În acest cadru, lucrarea investighează arhitectura [PAN](#) ca model [full-stack](#) bazat pe ecosistemul [TypeScript](#), în care straturile aplicației partajează tipuri și scheme de validare derivate dintr-o singură sursă de adevăr.

Aplicația utilizează randare pe server ([SSR](#)) cu [Angular](#) prin meta-framework-ul [Analog](#), comunicare tipizată client–server prin [tRPC](#) și persistența datelor prin [Prisma](#) și [CockroachDB](#). Organizarea proiectului într-un [monorepo](#) gestionat cu [Nx](#) impune o topologie aciclică a bibliotecilor, verificată la integrarea continuă ([CI](#)) prin regula `enforce-module-boundaries`, și permite eliminarea stratului de *repository* dintre procedurile [tRPC](#) și clientul [Prisma](#).

Implementarea este validată prin teste unitare cu [Vitest](#) și prin [testare cap-coadă](#) ([E2E](#)) cu [Playwright](#), însoțită de o evaluare empirică a latențelor stratului de persistență și a timpului de rulare [CI](#). Latența mediană măsurată a fost de 2,23 ms pentru [CockroachDB](#) și de 0,98 ms pentru [Redis](#), iar timpul mediu de rulare [CI](#) s-a redus cu aproximativ 25 % prin extragerea pașilor recurenți în acțiuni compozite reutilizabile. Rezultatele indică faptul că arhitectura [PAN](#) constituie un model coerent și extensibil pentru dezvoltarea aplicațiilor [web full-stack](#).

Cuprins

1	Introducere	1
1.1	Motivație	1
1.2	Scopul lucrării	1
1.3	Contribuții ale lucrării	2
2	Preliminarii	3
2.1	Motivația alegerilor tehnologice	3
2.2	Cadrul tehnologic	5
3	Implementare	8
3.1	Prezentare generală a implementării	8
3.2	Implementarea aplicației	9
3.3	Metodologie și evaluare	14
4	Concluzii	16
4.1	Direcții viitoare	18
	Anexe	19
	Anexa A Redactarea lucrării în L^AT_EX	19
A.1	Instrumente pentru redactare	19
A.2	Automatizarea compilării și reproductibilitatea	19
A.3	Compatibilitate multiplatformă	20
	Anexa B Manifest Arhitectural PAN	21
B.1	Definiție și principii structurale	21
B.2	Stiva tehnologică detaliată	22
B.3	Comparație cu arhitecturi alternative	23
B.4	Limitări și compromisuri	25
B.5	Direcții de evoluție arhitecturală	26
	Glosar de termeni	28
	Lista de acronime	32

1 Introducere

1.1 Motivație

Risipa alimentară reprezintă o problemă cu impact economic și social documentat la nivel global și european. Conform raportului *UNEP Food Waste Index 2024* [25], gospodăriile au generat în 2022 peste 631 de milioane de tone de alimente irosite — echivalentul a peste un miliard de mese aruncate zilnic și reprezentând 60 % din risipa alimentară totală la nivel mondial. La nivelul Uniunii Europene, Eurostat estimează că fiecare locuitor a generat în medie 132 de kilograme de risipă alimentară în 2022, din care 54 % provine din gospodării [16]. Parlamentul European identifică între cauzele principale ale risipei la nivel de consumator: cumpărăturile impulsive, lipsa unei planificări adecvate a meselor și gestionarea deficitară a inventarului casnic de alimente [20].

În paralel, aplicațiile [web](#) moderne ridică provocări tehnice semnificative, documentate atât în literatura academică, cât și în sondajele anuale ale comunității de dezvoltatori. Sondajul *State of JavaScript 2024* identifică *complexitatea excesivă, supraîncărcarea cu opțiuni și modificările care rup compatibilitatea* drept principalele puncte de durere semnalate de utilizatorii framework-urilor [frontend](#) [5]. Aceeași sursă indică o adopție în creștere a randării pe server (*SSR*) — 59 % dintre respondenți o folosesc în 2024 — și o consolidare a [TypeScript](#) ca limbaj standard pentru aplicațiile de mare anvergură: conform sondajului *JetBrains Developer Ecosystem 2024*, adopția [TypeScript](#) a crescut de la 12 % în 2017 la 35 % în 2024, iar 67 % dintre dezvoltatori scriu predominant [TypeScript](#) [18]. Aceste dificultăți se accentuează în aplicațiile [full-stack](#), unde integrarea deficitară a straturilor conduce la creșterea complexității și a costurilor de mentenanță pe termen lung.

Aceste două direcții — necesitatea unei soluții digitale pentru reducerea risipei alimentare la nivel casnic și exigențele tehnice ale aplicațiilor [web full-stack](#) moderne — converg în obiectivul prezentei lucrări.

1.2 Scopul lucrării

Mi-am propus să construiesc *Kookio* ca o aplicație [web](#) care abordează problema risipei alimentare la nivel casnic printr-o arhitectură [web](#) modernă, bazată pe ecosistemul [TypeScript](#). Arhitectura propusă integrează [frontend](#)-ul, [backend](#)-ul și stratul de persistență într-un proces unitar de dezvoltare. La nivel funcțional, aplicația acoperă gestiunea cămării, planificarea zilnică a meselor și derivarea automată a listei de cumpărături pentru utilizatori casnici. Aplicația servește drept studiu de caz pentru evaluarea arhitecturii [PAN](#) într-un context reprezentativ pentru aplicațiile [web full-stack](#) moderne.

În acest sens, lucrarea își propune:

- implementarea unui flux funcțional integrat pentru cămară, planificator zilnic și listă de cumpărături derivată, cu persistență într-o bază de date relațională;
- integrarea [frontend](#)-ului și [backend](#)-ului într-un proces de dezvoltare unitar, cu partajarea tipurilor și a mecanismelor de validare;
- analiza caracteristicilor și a proprietăților structurale ale arhitecturii [PAN](#) în ceea ce privește modularitatea (topologie aciclică între biblioteci) și mentenanța aplicației (absența unui strat de *repository*);
- construirea unui pipeline [CI/CD](#) reutilizabil, organizat pe [workflow-uri](#) și [composite actions](#), aliniat la țintele [Nx](#);
- validarea implementării prin teste unitare cu [Vitest](#) și prin [testare cap-coadă \(E2E\)](#) cu [Playwright](#), însoțită de o evaluare empirică a latențelor stratului de persistență și a timpului de rulare [CI](#).

1.3 Contribuții ale lucrării

1. **Aplicația [Kookio](#) implementată ca [produs minim viabil \(MVP\)](#):** aplicație [web full-stack](#) funcțională pentru utilizatori casnici, care acoperă integrat gestiunea cămării, planificarea zilnică a meselor, derivarea automată a listei de cumpărături și fluxul de *onboarding* (configurare profil), cu autentificare prin Firebase.
2. **Un model arhitectural [PAN](#) implementat:** [Analog](#) ([Angular](#) 20) cu randare pe server ([SSR](#)) peste [Nitro](#), [tRPC](#) pentru [API](#) tipizat și [Prisma](#) către [CockroachDB](#), organizat într-un [monorepo Nx](#) cu cinci grupuri de biblioteci și topologie aciclică aplicată la [CI](#) prin regula `enforce-module-boundaries`.
3. **Tipare de partajare a contractelor de date dintr-o sursă unică de adevăr:** schemele [Zod](#) sunt generate automat din `schema.prisma` prin biblioteca `prisma-zod-generator` [4], iar tipurile [TypeScript](#) se derivă prin compoziție (`.pick()`, `.omit()`, `.extend()`, `z.infer<>`) din aceste scheme; contractele rezultate sunt reutilizate în interfața de utilizator, în rutele de server și în teste, fără un strat de *repository* între procedurile [tRPC](#) și clientul [Prisma](#).
4. **Un flux automatizat [CI/CD](#) modular:** bazat pe [workflow-uri](#) reutilizabile și [composite actions](#) aliniate la țintele [Nx](#); reducerea timpului mediu de rulare este măsurată în Tabelul 1.
5. **Metodologie de testare pe două niveluri distincte:** testare unitară cu [Vitest](#) și [testare cap-coadă \(E2E\)](#) cu [Playwright](#), însoțită de practici concrete pentru pornirea la rece a

serverului [Vite](#) (pre-încălzire în `global-setup`) și pentru navigarea către rute protejate (`waitUntil: 'commit'`).

6. **Evaluare empirică a stivei tehnologice implementate:** măsurători de latență la stratul de persistență ([CockroachDB](#) și [Redis](#)) și a timpului de rulare [CI](#) înainte și după extragerea pașilor recurenți în [composite actions](#), sintetizate în Tabelul 1.

Aplicația este concepută ca un [MVP](#), care poate fi extins ulterior cu funcționalități precum recomandarea de rețete sau integrarea cu furnizori externi. Aceste extensii pot consolida contribuția aplicației la reducerea risipei alimentare prin diminuarea achizițiilor redundante. Detaliile privind implementarea sunt prezentate în Secțiunea 3, evaluarea empirică în Secțiunea 3.3, iar direcțiile concrete de evoluție în Secțiunea 4.1.

2 Preliminarii

Acest capitol prezintă contextul tehnologic al lucrării în două subsecțiuni complementare. Secțiunea 2.1 expune motivația alegerilor tehnologice care au condus la formularea arhitecturii [PAN](#), iar Secțiunea 2.2 descrie tehnic fiecare componentă a stivei și rolul ei în aplicația [Kookio](#).

2.1 Motivația alegerilor tehnologice

Pornind de la avantajele tipizării statice și de la consolidarea [TypeScript](#) ca limbaj standard pentru aplicațiile de mare anvergură — sondajul *JetBrains Developer Ecosystem 2024* indică o creștere a adopției de la 12 % în 2017 la 35 % în 2024, cu 67 % dintre dezvoltatori scriind predominant [TypeScript](#) [18] — în cadrul [Kookio](#) am explorat posibilitatea construirii unei aplicații complete, dintr-un capăt în celălalt, exclusiv pe [TypeScript](#). Această stivă tehnologică va fi denumită convențional, în prezenta lucrare, arhitectura [PAN](#) ([Prisma](#)–[Analog](#)–[Nx](#)).

Alegerea bazei de date

O primă provocare conceptuală a fost alegerea bazei de date. Istoric vorbind, [MongoDB](#) a fost pentru mulți ani favoritul implicit al dezvoltatorilor din ecosistemul JavaScript/[TypeScript](#), datorită stocării native în format JSON și a schemei flexibile care se mapează direct pe obiectele aplicației. Totuși, modelul de date al aplicației [Kookio](#) este intrinsec relațional: rețetele, ingredientele și utilizatorii sunt entități distincte legate prin relații de tip mai-mulți-la-mai-mulți, iar interogările compuse — legăturile de date, constrângerile de integritate referențială și tranzacțiile ACID — sunt esențiale pentru consistența datelor. Această caracteristică a modelului de date a făcut ca o bază relațională clasică să fie alegerea naturală pentru proiect.

Pentru abstractizarea limbajului bazei de date am ales [Prisma](#) [22] — o mapare obiect-relațională (ORM) care oferă siguranță a tipurilor de la capăt la capăt în [TypeScript](#), generând clientul de acces direct din schema declarativă. Ca implementare concretă a serviciului m-am orientat spre [CockroachDB](#) [3], o bază de date distribuită compatibilă cu protocolul de legătură [PostgreSQL](#) [17] — astfel încât [Prisma](#) și întregul ecosistem de unelte funcționează nealterat. [CockroachDB](#) adaugă peste [PostgreSQL](#) scalare orizontală nativă, consistență puternică prin algoritmul de consens Raft, precum și un nivel de servire fără infrastructură proprie gestionată (*serverless*), potrivit pentru un proiect academic care poate evolua către un mediu de producție.

Cadrul frontend și organizarea codului

Pentru interfața clientului, alegerea [framework](#)-ului [Angular](#) a venit firesc, dat fiind angajamentul față de [TypeScript](#) [9]. Pasul următor a fost identificarea unei opțiuni potrivite pentru codul ce urma să ruleze pe server. Aici am avut mai multe variante valide, majoritatea moștenitori conceptuali ai celebrei arhitecturi **MERN** (*MongoDB, Express, React, Node.js*), care în ultimii ani a migrat firesc către stive bazate pe [Next.js](#) și *Turborepo* — ambele dezvoltate de [Vercel](#). Totuși, această direcție m-ar fi împins către ecosistemul *React* și sintaxa *TSX*, abandonând decizia inițială de a folosi [Angular](#).

Pe parcursul cercetării am descoperit comunitatea [Analog](#) [6], un [meta-framework](#) care extinde [Angular](#) cu capabilități de server (randare pe server, rutare bazată pe fișiere, puncte finale API) — practic, echivalentul [Angular](#) al lui [Next.js](#). [Analog](#) beneficiază de sprijinul direct al comunității [Nx](#) [11], o unealtă de gestionare a [monorepo](#)-urilor dezvoltată de compania Nrwl, fondată de foști ingineri ai echipei [Angular](#) de la Google. Această filiație explică de ce [Nx](#) are, în esența sa, ADN [Angular](#) și de ce oferă suport de primă mână pentru proiecte [Analog](#): generatoare native, sistem de cache distribuit pentru sarcinile de construire și șabloane care accelerează crearea bibliotecilor specializate pe fiecare funcționalitate de bază a aplicației.

Astfel s-a conturat o paralelă structurală cu arhitectura **MEAN** (*MongoDB, Express, Angular, Node.js*) [19], arhitectura **PAN** reflectând maturizarea ecosistemului: [Analog](#) absoarbe rolurile pe care **MEAN** le distribuia între *Express* și [Angular](#), iar [Nx](#) adresează o preocupare arhitecturală — organizarea [monorepo](#) cu cache distribuit — care, în era **MEAN**, nu era încă subiect de discuție. O comparație detaliată între **MEAN**, stiva modernă *React* (**MERN** + [Next.js](#) + *Turborepo*) și **PAN** este prezentată în Anexa B (Figura 1).

Manifestul arhitectural complet, împreună cu avantajele comparative și direcțiile de evoluție, se află în **Anexa B**.

2.2 Cadrul tehnologic

2.2.1 Arhitecturi web moderne

Pentru o aplicație culinară în care prima impresie a utilizatorului depinde de un [HTML](#) populat încă de la prima vizită, modelul de randare pe client ([CSR](#)) al unui [SPA Angular](#) clasic nu era suficient. Cerințele de afișare rapidă, [SEO](#) pentru rețete publice și interactivitate ulterioară pe pagină au condus la o arhitectură care împarte sarcinile de [randare](#) între server și client [[6](#), [13](#)].

Strategiile de [randare](#) relevante pentru proiect sunt:

- [CSR](#) (randare pe client) — folosit în [Kookio](#) pentru ecranele autentificate (planificator, cămară), unde interactivitatea primează;
- [SSR](#) (randare pe server) — pentru pagini publice cu conținut dinamic (feed de rețete, profile), unde [HTML](#)-ul inițial trebuie să fie complet;
- [SSG](#) (generare statică de pagini) — pentru pagini cu conținut stabil (legal, atribuții), prerandate la build;
- **arhitecturi hibride** — combinarea celor trei pe rute diferite, în funcție de natura conținutului [[6](#)].

Am ales o arhitectură [full-stack](#) hibridă în care [frontend](#)-ul și [backend](#)-ul rulează în aceeași bază de cod [TypeScript](#). Decizia este motivată practic: tipurile și schemele de validare ([tRPC](#) cu validare prin scheme [Zod](#)) sunt partajate la nivel de modul, iar un singur pas de construcție livrează ambele straturi [[26](#), [2](#)].

2.2.2 Angular și randare server-side

Am ales [Angular](#) ca [framework](#) de [frontend](#) pentru că ecosistemul său oferă în mod nativ piesele de care aveam nevoie pentru [Kookio](#): tipizare strictă, componente *standalone*, [Angular](#) Signals (semnale reactive) și setul de componente [Angular](#) Material/CDK, consistent în privința accesibilității [[9](#), [8](#)].

Suportul pentru [SSR](#) din [Kookio](#) provine din [Analog](#), un [meta-framework](#) care extinde [Angular](#) cu:

- [SSR](#) (randare pe server) și [SSG](#) (generare statică de pagini) configurabile per rută,
- rutare bazată pe structura de fișiere,
- integrarea logicii de server ([Nitro](#), [tRPC](#)) în același proiect [[6](#), [7](#), [13](#)].

Pentru o pagină de rețetă publică, [SSR](#) face diferența între un document [HTML](#) minimal până la [hidratare \(hydration\)](#) și un [HTML](#) populat încă de la prima cerere — important atât pentru utilizator, cât și pentru indexarea [SEO](#) pe care se sprijină distribuția organică a conținutului [6].

2.2.3 Monorepo și orchestrare cu Nx

Am organizat [Kookio](#) ca [monorepo](#) folosind [Nx](#): o singură aplicație (apps/web) și un set de biblioteci segmentate pe responsabilități (core, feature, prisma, server, shared) [11].

Bibliotecile sunt grupate pe roluri:

- core/ — configurație, logare, securitate (preocupări transversale),
- feature/ — fluxuri de utilizator (auth, shell),
- prisma/client — schema, clientul generat, schemele [Zod](#),
- server/ — context [tRPC](#), middleware-uri, funcții utilitare de runtime server,
- shared/ — [UI](#), utilitare, date simulate (*mock*).

Peste această structură, [Nx](#) oferă două mecanisme esențiale folosite sistematic în proiect: `nx affected` rulează numai sarcinile (`lint`, `test`, `build`) atinse de modificările curente, iar regula `enforce-module-boundaries` respinge la [CI](#) importurile care încalcă topologia de biblioteci [11].

2.2.4 tRPC și partajarea tipurilor

Comunicarea client–server din [Kookio](#) folosește [tRPC](#) (*TypeScript Remote Procedure Call* — apeluri de proceduri la distanță tipizate pentru [TypeScript](#)): procedurile sunt declarate o singură dată, în [TypeScript](#), pe server, iar clientul [Angular](#) le invocă direct, fără un strat suplimentar de definiții [API](#) [26].

Avantajele concrete pe care le-am observat:

- inferența completă [TypeScript](#) elimină [DTO-uri](#) scrise manual care apar în arhitecturile REST clasice,
- validarea de intrare se face prin scheme [Zod](#) generate automat din schema [Prisma](#) [2, 4],
- refactorările prin redenumire de câmpuri propagă instantaneu pe ambele straturi.

Pentru [Kookio](#) aceasta s-a tradus, în practică, în eliminarea aproape completă a verificărilor de tip la frontiera client–server: contractul este compilatorul.

2.2.5 Persistență: Prisma și CockroachDB

Pentru persistență am ales **CockroachDB** ca bază de date și **Prisma** drept **ORM** [22, 3]. **CockroachDB** este compatibil cu protocolul de legătură **PostgreSQL**, ceea ce înseamnă că ecosistemul de unelte **SQL** (drive, clienți `psql`, **Prisma**) funcționează fără adaptări, oferind în plus capacități native de scalare orizontală. **PostgreSQL** standard se concentrează pe replicare verticală, în timp ce **CockroachDB** adaugă peste protocol scalare orizontală nativă [3].

Prisma acoperă, în proiect, trei responsabilități: declararea schemei în `schema.prisma`, generarea unui client **TypeScript** cu tipuri exacte pentru fiecare model și gestionarea migrațiilor versionate aplicate la `deploy` [22].

La generarea schemelor **Zod** folosesc `prisma-zod-generator`, care expune o copie validabilă a fiecărui model — sursa de adevăr este schema **Prisma**, iar straturile **API** și **UI** consumă derivate ale ei [4].

2.2.6 Testare automată: Vitest și Playwright

Suita de testare din **Kookio** are două straturi distincte: teste unitare cu **Vitest** pentru logica izolată și **testare cap-coadă** (E2E) cu **Playwright** pentru scenariile complete printr-un browser real [15, 10].

Am ales **Vitest** pentru că rulează deja pe **Vite** (același bundler ca aplicația principală), ceea ce înseamnă că nu există configurări duplicate sau divergențe între `nx test` și `nx build` [15, 14].

Playwright acoperă fluxurile critice: autentificare, finalizare *onboarding*, adăugare în cămară, vizualizare planificator. Suita rulează în **CI** pe fiecare **PR** către `test` înainte de promovarea în `main` [10].

2.2.7 Managementul configurației și mediilor

Variabilele de mediu (chei **Firebase**, **URL**-uri **Redis**, secrete **Vercel**) sunt definite o singură dată în panoul **Vercel**, separat pe medii (`development`, `preview`, `production`) [29]. Această centralizare elimină fișierele `.env` adăugate accidental în istoricul **Git** și menține sincronizate toate ramurile.

Local, sincronizarea se face cu `vercel env pull`, care scrie un `.env.local` (exclus din versionare prin `.gitignore`) cu valorile mediului curent — niciun secret nu este versionat în **Git** [29].

3 Implementare

3.1 Prezentare generală a implementării

Capitolul precedent a stabilit alegerile tehnologice care formează arhitectura **PAN** (manifestul complet în Anexa B). În cele ce urmează descriu cum se traduce acest tipar în structura efectivă a **repository**-ului: o singură aplicație **Analog** sub `apps/`, un set de biblioteci **Nx** grupate pe responsabilități sub `libs/` și un lanț unitar de execuție prin care o cerere ajunge de la runtime-ul server-side al **Analog** către straturile **trPC** și **Prisma**.

3.1.1 Structura monorepo-ului Nx

```
kookio/
├─ apps/
│   ├─ web/                                # aplicație Analog (Angular 20 + SSR),
│   │                                     rulează atât în browser cât și ca
│   │                                     runtime server (Nitro)
│   └─ web-e2e/                            # suită Playwright pentru fluxurile
│                                       critice (login, onboarding, pantry,
│                                       planificator)
├─ libs/
│   ├─ core/                              # config, logging, security
│   │                                     (preocupări trans-feature)
│   ├─ feature/                            # fluxuri vizibile utilizatorului
│   │   ├─ auth/                          # login, înregistrare, sesiune
│   │   ├─ onboarding/                   # pași de configurare profil
│   │   ├─ pantry/                       # gestiune cămară
│   │   ├─ plan/                         # planificator zilnic
│   │   ├─ recipes/                     # feed și detaliu rețete
│   │   ├─ groceries/                   # listă derivată de cumpărături
│   │   ├─ saved/                       # colecții salvate
│   │   ├─ dashboard/                   # ecran principal autentificat
│   │   └─ shell/                       # layout, topbar, navigare
│   ├─ prisma/client                     # schemă Prisma, migrații,
│   │                                     scheme Zod generate, tipuri derivate
│   ├─ server/                           # auth, logger, trpc
│   │                                     (helperi de runtime server)
│   └─ shared/                           # ui, utility, mock
│                                       (componente, helperi, fixtures)
├─ nx.json                               # configurație globală Nx
├─ package.json                           # manifestul principal al proiectului
├─ tsconfig.base.json                     # config TypeScript + alias-urile
├─ vercel.json                            # rute, headers și build pentru Vercel
└─ vitest.workspace.ts                    # workspace Vitest (toate proiectele)
```

În practică, o modificare în `libs/feature/pantry` declanșează doar verificările pentru biblioteca afectată și pentru modulele care depind direct de ea (via `nx affected`), nu pentru întregul [repository](#) [11]. Topologia dintre grupuri (`feature/` nu importă din `feature/`, `shared/` nu depinde de nimic intern) este aplicată automat de regula `enforce-module-boundaries`, verificată la [CI](#).

3.1.2 Fluxul aplicației cap-coadă

La nivel de ansamblu, o cerere parcurge trei straturi: *Analog* (*UI* și *SSR*) → *tRPC* (*API* tipizat) → *Prisma* (*acces la date*) [6, 26, 22]. Tranzițiile dintre straturi nu trec printr-un strat suplimentar de serializare manuală — tipurile [TypeScript](#) sunt partajate de la capăt la capăt, iar validarea de intrare la frontiera *tRPC* folosește scheme [Zod](#) derivate din schema [Prisma](#) [26, 2, 4]. Detalierea pe etape a unei cereri concrete (de la *SSR* inițial până la apelurile ulterioare din browser) se găsește în Secțiunea 3.2.8.

3.2 Implementarea aplicației

Dacă secțiunea anterioară a justificat alegerile tehnologice, cea de față arată cum se vede fiecare alegere în repository: ce fișiere o materializează, ce convenții o aplică și ce constrângeri concrete a impus la nivelul codului. Pentru fiecare tehnologie, accentul nu mai cade pe *de ce* a fost aleasă, ci pe *ce a rezultat efectiv* odată integrată în [Kookio](#).

3.2.1 Arhitecturi web moderne

Configurarea de [randare](#) a [Kookio](#) este concentrată în `apps/web/vite.config.ts`: plugin-ul `analog` este apelat cu `ssr: true` și o singură rută declarată în `prerender.routes`, anume `/about`. Practic, *SSR* este modul implicit pentru toate rutele aplicației, iar *SSG* este folosit punctual pentru singurele pagini al căror conținut nu depinde de cerere [6].

Pe ramura *SSR*, distincția dintre rutele publice (`/recipes`, `/legal/attributions`) și cele autentificate (`/pantry`, `/plan`) nu se manifestă în strategia de [randare](#), ci în ceea ce execută serverul înainte de a servi pagina: rutele autentificate trec printr-un *guard* (interceptor de rută) care emite o redirectionare către `/login` dacă sesiunea Firebase lipsește, vizibilă pentru [Playwright](#) drept `ERR_ABORTED` pe Chromium și gestionată prin `waitUntil: 'commit'` în testele [E2E](#).

3.2.2 Topologia bibliotecilor și absența unui strat de repository

Bibliotecile *feature* ale aplicației sunt felii verticale (*vertical slices*): fiecare reprezintă un flux utilizator complet (`auth`, `onboarding`, `pantry`, `plan`, `recipes`) și conține componentele, ma-

gazinele de stare (*store-uri* NgRx Signals) și serviciile de care depinde acel flux. Topologia impusă de `enforce-module-boundaries` este aciclică: `apps/` → `libs/feature/` → `libs/core/`, `libs/server/`, `libs/shared/`, `libs/prisma/`, iar `libs/feature/X` nu poate importa din `libs/feature/Y`. Comunicarea între *feature-uri* se face prin magazine de stare `providedIn: 'root'` sau prin procedurile `tRPC` expuse de `libs/server/trpc` [11, 12, 26].

Codul `backend` și cel `frontend` coexistă în același `repository`: clientul `Prisma` și schemele `Zod` generate sunt localizate în `libs/prisma/client`, iar tipurile lor sunt importate direct în componentele `Angular` [4]. Nu există un strat clasic de tip *repository* orientat-obiect între procedurile `tRPC` și `Prisma`; dispecerii de rute apelează direct `ctx.prisma`, iar tranzacțiile multi-pas folosesc `prisma.$transaction` [26, 24].

3.2.3 Angular și randare server-side

Rutele aplicației sunt definite pe baza structurii de fișiere (*filesystem-based*) sub `apps/web/src/app/pages/`: un fișier `about.page.ts` expune `/about`, `cook.[recipeId].page.ts` expune o rută dinamică cu parametru, iar un `cook.[recipeId].server.ts` colocat realizează încărcarea de date înainte ca `Angular` să randeze componenta [6, 7]. Fiecare `*.page.ts` este export implicit (*default export*) și este încărcat la cerere (*lazy loading*) de către dispecerul de rute `Analog`; nu există un fișier centralizat de rute.

Configurarea `SSR` la nivel de aplicație este minimală: `app.config.ts` include `provideFileRouter()`, `provideHttpClient(...)` și `provideClientHydration(...)`, ultimul rerulând prin opțiunea `withEventReplay()` evenimentele utilizator capturate în intervalul dintre `SSR` și *hidratare (hydration)* [6]. Fiecare import adăugat în `app.config.ts` are însă un cost amplificat de `SSR`: modulul este evaluat pe traseul critic al fiecărei rute, iar fișierele agregatoare de tip *barrel* (re-exporturi de tipul `export *`) nu sunt eliminate de `Vite` în mediul de dezvoltare. Constrângerea practică este să se importe direct, prin sub-căi, doar tokenii efectiv utilizați, lăsând componentele grele (`Material datepicker`, `autocomplete`) să fie încărcate de la nivelul paginii care le folosește.

3.2.4 Gestionarea datelor cu Prisma și CockroachDB

Schema bazei de date se află la `libs/prisma/client/schema.prisma`, alături de migrațiile gestionate de `Prisma` și de generatorul `prisma-zod-generator`, care produce o copie validabilă a fiecărui model în formă `Zod` [22, 4]. Tipurile derivate (seturi inițiale de date, intrări de formular, *payload-uri* pentru rute `tRPC`) sunt construite prin compoziție — `.pick()`, `.omit()`, `.extend()` — direct peste schemele generate, astfel încât o redenumire de câmp în `schema.prisma` să se propage automat în consumatori [4].

Două decizii structurale susțin acest aranjament. Întâi, indexul public @kookio/prisma-client este sigur pentru execuție în browser: exportă numai tipuri și scheme [Zod](#), iar instanța PrismaClient este izolată sub sub-calea @kookio/prisma-client/server, iar importul acesteia este blocat de ESLint în codul de [frontend](#) [22]. Apoi, instanța [Prisma](#) este extinsă cu o reluare automată la conflict (*retry-on-conflict*) pentru codurile de eroare *serializable* specifice [CockroachDB](#) (P2034, 40001); toate operațiile de scriere (create, update, upsert, delete și variantele *Many) trec prin acest *wrapper* (înveliș de cod), iar tranzațiile multi-pas sunt declarate idempotente pentru a permite reluarea automată [24, 3].

Constrângerile de unicitate compusă (de exemplu @@unique([userId, date, slotType]) pe MealSlot, sau perechea userId-ingredientCatalogId pe PantryItem) sunt declarate în schema.prisma și aplicate la nivelul bazei de date, iar [Prisma](#) generează automat formele cu cheie compusă (*compound key*, de tipul where: { userId_ingredientCatalogId: { ...} }) pentru interogările de căutare aferente [22].

3.2.5 Comunicarea client-server cu tRPC

Dispecerii de rute [tRPC](#) sunt definiți sub libs/server/trpc/src/lib/routers/, câte un dispecer de rute per domeniu (auth, pantry, onboarding, meal-plan, recipes, cooking-session ș.a.). Fiecare procedură este compusă dintr-unul din trei *constructori* declarați în libs/server/trpc/src/lib/middlewares/auth.ts: publicProcedure (expusă anonim, fără autentificare), protectedProcedure (necesită sesiune Firebase verificată în createContext) și onboardedProcedure (necesită în plus onboardingDone === true pe utilizator) [26, 27]. Convenția este una de structură: o căutare cu grep prin constructorul folosit într-un dispecer de rute arată imediat ce interfață expune punctul de acces, fără a fi nevoie să se inspecteze fiecare strat de *middleware* compus local.

Pe partea de client, instanța [tRPC](#) este creată în apps/web/src/trpc-client.ts cu un authAwareFetch dedicat care (i) atașează *ID token*-ul Firebase ca antet HTTP Authorization, (ii) împrăștează proactiv tokenul când expiră în mai puțin de cinci minute, (iii) rezolvă [URL](#)-urile relative la formă absolută în context [SSR](#) (Node.js nu are o bază implicită pentru fetch('/api/...')). Procedurile [tRPC](#) returnează Observable-uri RxJS în această configurație, iar consumatorii din [Angular](#) aleg între from(...).pipe(...) pentru flux reactiv și await firstValueFrom(from(...)) pentru un singur rezultat [26].

3.2.6 Rute h3 auxiliare și infrastructura Redis

Pe lângă handler-ul [tRPC](#) (apps/web/src/server/routes/api/trpc/[trpc].ts), aplicația expune două rute h3 suplimentare: /api/health pentru verificarea de stare (*healthcheck*) și /api/internal/bench-db pentru benchmark-ul de latență al [CockroachDB](#)

(folosit în secțiunea de evaluare). Un singur *middleware* `h3` (`apps/web/src/server/middleware/security.ts`) acoperă cerințele transversale — antete de securitate și 404 explicit pentru `/assets/*` negăsite, pentru a evita situația în care SSR-ul ar returna 200 cu HTML pentru un fișier de tip *bundle* înlocuit între deploy-uri.

Componenta Redis se află în `apps/web/src/server/infra/` și este proiectată să tolereze indisponibilitatea. `getRedis()` este un singleton cu *circuit breaker* de 60 de secunde: după o conexiune eșuată, apelurile următoare eșuează rapid (*fail-fast*) în loc să recreeze clientul [28]. Pe ramura de citire (`groceries.list`), o funcție utilitară captează silențios eșecurile Redis și returnează mulțimea goală, astfel încât pagina să se randeze fără starea «cumpărat»; pe ramura de scriere, răspunsul include un câmp `persisted: false` dacă Redis este indisponibil, permițând clientului să păstreze comutarea optimistă (*optimistic update*) și să afișeze o notificare. Limitarea ratei (*rate-limiting*, `rateLimitFixedWindow`) folosește o fereastră fixă `INCR + EXPIRE` și este aplicată momentan doar pe punctul final de benchmark.

3.2.7 Jurnalizare și observabilitate

Stratul de jurnalizare (*logging*) este izolat în `libs/server/logger/`: o singură instanță Pino este creată cu un transport dependent de mediu — `pino-pretty` cu *colorize* și marcaj de timp scurt în dezvoltare, JSON brut în producție (Vercel) [1]. Pe ramura de producție, instanța aplică o listă explicită de câmpuri redactate (*redact list*: `password, *.token, authorization`) pentru a împiedica scurgerea credențialelor în jurnale (*log-uri*).

Pe lângă instanța globală de jurnalizare, `createLogger(namespace, bindings)` returnează un *logger* copil cu o etichetă de *namespace* (`ns: 'pantry', ns: 'auth'`) care însoțește toate intrările emise de acea componentă, iar `createAudit(bindings)` returnează un logger specializat pentru evenimente de audit (cu indicatorul `audit: true` aplicat automat). Mesajele sunt întotdeauna structurate — `logger.info({ userId, action }, 'message')` — nu interpolări de șir, pentru ca log-urile produse să fie filtrabile direct pe platforma de agregare.

3.2.8 Fluxul de execuție al unei cereri

Detalierea fluxului rezumat în Secțiunea 3.1.2 se desfășoară pe trei etape.

Etapa 1 (randare inițială). Cererea HTTP ajunge la runtime-ul server-side al Analog (bazat pe Nitro), iar instanța Angular este executată pe server pentru a produce HTML-ul inițial [6, 13].

Etapa 2 (interogarea persistenței). Componentele Angular declanșează apeluri către procedurile tRPC ale stratului server-side; fiecare procedură își validează intrările cu o schemă Zod derivată din schema Prisma și folosește `ctx.prisma` pentru a interoga CockroachDB [26, 2, 4,

21]. Rezultatele se propagă înapoi prin stratul [tRPC](#) către componentele care participă la randarea inițială.

Etapa 3 (hidratare și interacțiune ulterioară). [HTML](#)-ul randat ajunge la browser, iar instanța [Angular](#) preia controlul prin [hidratare \(hydration\)](#) — arborele [DOM](#) produs pe server este reutilizat, iar evenimentele capturate în interval sunt rerulate de `withEventReplay()` [6]. De aici, interacțiunile ulterioare apelează direct procedurile [tRPC](#) din browser, fără o nouă [randare](#) pe server [26].

3.2.9 Testare și validare

Suita de [Vitest](#) este orchestrată de `vitest.workspace.ts` la rădăcina [repository](#)-ului, care colectează automat toate fișierele `vite.config.*` și `vitest.config.*` din proiectele [Nx](#) [15, 14]. Fișierele de specificații (*spec-uri*, `*.spec.ts`) sunt colocate lângă codul testat, atât în bibliotecile de *feature*, cât și în bibliotecile de server (de exemplu `pantry.router.spec.ts` alături de `pantry.router.ts`). Granularitatea este principiul director: `nx affected -t test` rulează numai *spec-urile* atinse de modificările curente.

Suita [Playwright](#) se află la `apps/web-e2e/` și are propriul `global-setup.ts` care preîncălzește toate rutele aplicației înainte ca primul proces de lucru (*worker*) să pornească — măsură necesară pentru a evita constrângerile de timp la prima pornire pe [Vite](#) în mediul de dezvoltare [10]. Fluxurile critice (autentificare, înregistrare, finalizare *onboarding*, adăugare în cămară, vizualizare planificator) parcurg interfața în Chromium; rutele protejate folosesc `waitUntil: 'commit'` pentru a evita eroarea `ERR_ABORTED` produsă de redirectionările [Angular](#) în timpul [hidratare \(hydration\)](#).

Disciplina cea mai vizibilă în practică se aplică la refactorizările care modifică dependențele injectate într-o componentă partajată: testele bibliotecii respective continuă să treacă (stub-urile locale — *stubs*, înlocuitori de test — sunt actualizate), dar *spec-urile* modulelor dependente pot eșua cu `NullInjectorError`. Pentru a detecta aceste regresii, se rulează `nx affected -t test`, care urmărește graful de dependențe [Nx](#) și rulează tot ce este afectat [11].

3.2.10 Dezvoltare și deploy

Configurarea mediilor este centralizată în panoul [Vercel](#), separat pe *production*, *preview* și *development* [29]. `Local, vercel env pull` sincronizează valorile mediului curent într-un `.env.local` (exclus din versionare prin `.gitignore`), eliminând necesitatea ca vreun secret să fie versionat în Git.

Promovarea între medii respectă fluxul `dev → test → main`. Ramura *dev* alimentează mediul de previzualizare (*Preview*) de dezvoltare; ramura *test* declanșează suită [E2E](#) pe mediul

de previzualizare de testare înaintea promovării în `main`, iar ramura `main` alimentează publicarea (*deploy*) de producție în regiunea `fra1` (Frankfurt) [29].

Build-ul de producție rulează `nx build web` cu `NODE_OPTIONS=-max-old-space-size=6144`. Limita implicită a heap-ului V8 este insuficientă pentru *bundling*-ul integral al stivei [Angular Material](#) în context [Nitro](#): fazele de transformare a arborelui sintactic abstract (AST) și de generare a hărților sursă (*source maps*) acumulează volume mari de obiecte temporare, iar build-ul eșuează cu JavaScript `heap out of memory` înainte de finalizare. Ridicarea limitei la 6 GB asigură marja necesară pentru ambele faze. [Prisma Client](#) este regenerat în fluxul automatizat (*pipeline*) — ținta generate este dependență a țintei build —, evitând astfel incompatibilitățile de binar (*binary*) între platformele [darwin-arm64](#) și [rhel-openssl-3.0.x](#) [23].

3.3 Metodologie și evaluare

3.3.1 Setare experimentală

Toate măsurătorile au fost realizate în mediul *Preview* al [Vercel](#) (regiunea `fra1`, Frankfurt), cu [CockroachDB Serverless](#) și [Redis Cloud](#) în aceeași regiune; stiva rulează [Analog](#) peste [Nitro](#), cu [SSR](#) la prima cerere și [hidratare \(hydration\)](#) la client. Setul de date provine din seturi deterministe de date de test (*fixture-uri*, `libs/shared/mock/`) și din scripturile de inițializare (*seed*, `libs/prisma/client/seed-data/catalog.json`, `recipes.json`).

Toate procedurile de măsurare sunt versionate în [repository](#)-ul lucrării: ruta de benchmark este definită în `apps/web/src/server/routes/api/internal/bench-db.get.ts`, fluxurile de [CI](#) sunt în `.github/workflows/`, iar configurația suitei [E2E](#) este în `apps/web-e2e/playwright.config.ts`. Rezultatele de latență pot fi reproduse executând `npm run bench:preview local` (cu credențialele de acces [Vercel](#) corespunzătoare), iar metricile de [CI](#) și [E2E](#) prin declanșarea fluxului automatizat pe orice ramură de dezvoltare.

3.3.2 Indicatori și proceduri

Timpul de rulare [CI](#) (integrare continuă). Se compară timpul median de execuție al [workflow](#)-ului [CI](#) înainte și după extragerea pașilor recurenți (`setup`, `build`, `test`) în acțiuni compozite (*composite actions*) reutilizabile; rulările de referință sunt cele de pe ramurile `test` și `main` din ultima săptămână de dezvoltare.

Latența la stratul de persistență. Ruta internă `/api/internal/bench-db` execută $N = 100$ interogări succesive de citire către [CockroachDB](#), respectiv [Redis](#); se raportează media

aritmetică, percentila 50 ([p50](#)) și percentila 95 ([p95](#)), separând explicit prima cerere — rece (*cold*) — de cele subsecvente — calde (*warm*).

Fiabilitatea testelor end-to-end ([E2E](#)). Rata de trecere a suitei [Playwright](#) pe Chromium, măsurată pe ultimele zece rulări de pe ramura `test`. Browsersle Firefox și WebKit sunt configurate, dar nu rulează sistematic în [CI](#) (sunt acoperite în execuții manuale înaintea promovărilor în `main`).

3.3.3 Rezultate

Tabelul 1: Sinteza rezultatelor pe cele trei dimensiuni măsurate.

Indicator	Valoare
<i>Latența la stratul de persistență ($N = 100$ citiri)</i>	
CockroachDB — medie	2,55 ms
CockroachDB — p50	2,23 ms
CockroachDB — p95	3,52 ms
Redis — medie	1,20 ms
Redis — p50	0,98 ms
Redis — p95	1,52 ms
<i>Timpul de rulare CI (mediană)</i>	
Configurație monolitică	4 min
După extragerea în acțiuni compozite	3 min
Reducere relativă	$\approx 25\%$
<i>Fiabilitatea testelor E2E</i>	
Rată de trecere Chromium (10 rulări)	100 %

Rezultatele sintetizate în Tabelul 1 confirmă, în limitele explicate în Secțiunea 3.3.4, viabilitatea arhitecturii [PAN](#) pe cele trei axe de evaluare: latențe milisecundice la stratul de persistență, reducere semnificativă a timpului de rulare [CI](#) prin extragerea pașilor recurenți, și fiabilitate completă a fluxurilor critice end-to-end pe browserul de referință.

3.3.4 Discuție și limitări

Toate măsurătorile provin din mediul *Preview*, în regiunea `fra1`, fără trafic concurent — așadar nu surprind comportamentul stivei sub încărcare sau în regim multi-regional. Suita **E2E** rulează doar pe Chromium în **CI**, iar matricile de testare Firefox și WebKit sunt acoperite manual; o promovare automatizată a celor două ar mări încrederea în compatibilitatea inter-browser (*cross-browser*). În plus, atât **CockroachDB**, cât și **Redis** rulează pe planurile gratuite (cu cote de utilizare și ciclu de *auto-suspendare* pentru **Redis**) și sunt măsurate fără trafic concurent, ceea ce face latențele raportate reprezentative pentru un regim singular, nu pentru producție la scară. Generalizarea acestor rezultate cere teste de încărcare dedicate și măsurători repetate într-un mediu de producție stabil.

4 Concluzii

Am proiectat și implementat **Kookio**, o aplicație **web full-stack** construită integral în ecosistemul **TypeScript** pe arhitectura **PAN**. Pornind de la preferința pentru **TypeScript** motivată în Capitolul 2, am tradus această preferință într-o stivă concretă: o singură aplicație **Analog** sub `apps/web`, cinci grupuri de biblioteci **Nx** (`core`, `feature`, `prisma`, `server`, `shared`) cu topologie aciclică aplicată de `enforce-module-boundaries` la **CI**, și un flux cap-coadă **Analog** → **tRPC** → **Prisma** descris în Secțiunea 3.1.2.

Adoptarea arhitecturii **PAN** a permis eliminarea straturilor clasice de adaptare între **frontend** și **backend**. Procedurile **tRPC** apelează direct `ctx.prisma` fără un strat de *repository* (Secțiunea 3.2.2), iar schemele **Zod** consumate de **Angular** sunt derivate din `schema.prisma` prin `prisma-zod-generator` — o redenumire pe un câmp se propagă la compilare în toți consumatori. Indexul public `@kookio/prisma-client` este sigur pentru execuție în browser (exportă numai tipuri și scheme **Zod**), iar instanța `PrismaClient` este izolată sub calea `@kookio/prisma-client/server`, blocată de o regulă ESLint dedicată în codul de **frontend**.

Combinăția dintre **Angular** și **Analog** a livrat **SSR** ca mod implicit pentru toate rutele aplicației și **SSG** doar pentru ruta `/about`, configurate într-un singur loc (`apps/web/vite.config.ts`). Configurația **SSR** la nivel de aplicație este minimală — `provideFileRouter()`, `provideHttpClient(...)` și `provideClientHydration(withEventReplay())` —, iar disciplina identificată în practică (Secțiunea 3.2.3) este că fiecare import adăugat în `app.config.ts` are un cost amplificat pe traseul critic al fiecărei rute, motiv pentru care componentele Material grele rămân importate de la nivelul paginii care le folosește.

Pe stratul de persistență, **Prisma** peste **CockroachDB** a oferit beneficii observabile direct la nivelul codului. Generarea automată a clientului **TypeScript** și a schemelor **Zod** din `schema.prisma` reduce contractul de date la o singură sursă de adevăr; constrângerile

compuse declarate ca `@@unique` (`MealSlot` pe `userId-date-slotType`, `PantryItem` pe `userId-ingredientCatalogId`) sunt aplicate la nivelul bazei de date, iar interogările de căutare aferente folosesc forma cu cheie compusă generată automat. Pentru a tolera reluările pe care [CockroachDB](#) le poate cere sub izolarea *serializable* implicită, instanța [Prisma](#) este extinsă cu o reluare automată la conflict (*retry-on-conflict*) pentru codurile P2034 și 40001; toate operațiile de scriere (`create`, `update`, `upsert`, `delete` și variantele **Many*) trec prin acest *wrapper* (înveliș de cod), declarat o singură dată.

Procesul de testare a confirmat valoarea combinării a două niveluri distincte. Suita [Vitest](#) (orchestrată de `vitest.workspace.ts` la rădăcina [repository](#)-ului) acoperă logica izolată prin fișiere de specificații colocate cu codul testat, iar `nx affected -t test` rulează doar speciile atinse de modificarea curentă. Suita [Playwright](#) de la `apps/web-e2e/` parcurge fluxurile critice (autentificare, *onboarding*, cămară, planificator) prin Chromium și a obținut o rată de trecere de 100 % pe cele zece rulări de referință (Tabelul 1). Două practici s-au consolidat din ciclurile concrete de [CI](#): pre-încălzirea rutelor în `global-setup.ts` înainte ca primul proces de lucru (*worker*) să pornească, și `waitUntil: 'commit'` pentru rutele protejate, ca răspuns la eroarea `ERR_ABORTED` produsă de redirecționările [Angular](#) în timpul hidratării.

Din perspectiva obiectivelor declarate în Capitolul 1, am livrat un [produs minim viabil \(MVP\)](#) care acoperă fluxurile centrale ale aplicației: gestiunea cămării, planificarea zilnică a meselor, derivarea listei de cumpărături și *onboarding*-ul utilizatorului. Structura de [monorepo](#) pe felii verticale (`libs/feature/auth`, `onboarding`, `pantry`, `plan`, `recipes...`), în care `libs/feature/X` nu poate importa din `libs/feature/Y` — regulă verificată la [CI](#) prin `enforce-module-boundaries` —, permite extinderea cu fluxuri noi ca biblioteci adiționale, fără modificări structurale ale aplicației existente.

În ansamblu, [Kookio](#) arată că arhitectura [PAN](#) poate funcționa coerent pentru o aplicație [web full-stack](#) reală: latențele măsurate pe ruta `/api/internal/bench-db` (mediană 2,23 ms pentru [CockroachDB](#), 0,98 ms pentru [Redis](#) — Tabelul 1), timpul median de [CI](#) redus de la 4 la 3 minute după extragerea pașilor recurenți în acțiuni compozite (*composite actions*) și rata de trecere de 100 % a suitei [E2E](#) pe Chromium susțin acest model în limitele explicate în Secțiunea 3.3.4 — mediul *Preview*, regiune unică `fra1`, planuri gratuite pentru [CockroachDB](#) și [Redis](#), matricea de testare inter-browser (*cross-browser*) acoperită manual.

Contribuția principală a prezentei lucrări este propunerea și validarea empirică a arhitecturii [PAN](#) ca model coerent pentru aplicații [web full-stack](#) în ecosistemul [Angular/TypeScript](#), demonstrată printr-un studiu de caz funcțional în domeniul reducerii risipei alimentare la nivel casnic.

4.1 Direcții viitoare

Direcțiile naturale de evoluție pornesc de la limitele identificate în Secțiunea 3.3.4. La nivelul infrastructurii, promovarea [CockroachDB](#)-ului și a [Redis](#)-ului pe planuri plătite, alături de activarea unei a doua regiuni dincolo de fra1, ar permite caracterizarea stivei sub trafic concurent. În domeniul testării, includerea sistematică a matricilor de testare Firefox și WebKit în [CI](#) ar consolida încrederea în compatibilitatea inter-browser (*cross-browser*) dincolo de Chromium. Pe planul funcționalității, infrastructura [Redis](#) existentă (folosită deocamdată pentru marcarea articolelor cumpărate din lista de cumpărături și pentru limitarea ratei pe ruta de benchmark — Secțiunea 3.2.6) poate fi extinsă cu un strat de *caching* aplicativ pentru paginile publice, iar un modul de recomandare a rețetelor bazat pe inventarul din cămară și pe preferințele utilizatorului ar valorifica fluxul de *onboarding* deja implementat.

Topologia aciclică a [monorepo](#)-ului face ca aceste extensii să poată fi realizate incremental: o nouă bibliotecă în `libs/feature/` cu propriul dispecer de rute [tRPC](#) se adaugă fără a atinge restul aplicației.

Anexa A Redactarea lucrării în $\text{\LaTeX}$

A.1 Instrumente pentru redactare

Lucrarea a fost redactată cu motorul [LuaLaTeX](#) (distribuit prin TeX Live 2025), folosind fonturi [Unicode](#) și pachetul [biblatex](#) cu backend-ul biber pentru gestionarea bibliografiei. Alegerea [LuaLaTeX](#) peste XeLaTeX a fost motivată de suportul nativ pentru extensii prin Lua și de gestionarea îmbunătățită a fonturilor OpenType, relevantă pentru caracterele [Unicode](#) speciale folosite în redarea structurii de fișiere.

Pe partea de toolchain de dezvoltare, lucrarea a fost editată în Visual Studio Code cu extensia *LaTeX Workshop*, configurată să ruleze pipeline-ul `lualatex → makeglossaries → biber → lualatex → lualatex` la fiecare invocare a fluxului automatizat.

Structura sursei este modulară: fiecare capitol este un fișier `.tex` separat, inclus prin `\input{}` în documentul principal `kookio_thesis.tex`. Imaginile sunt salvate în `images/`, iar citările se realizează prin macrocomanda `\cite{cheie}` pe baza intrărilor din fișierul `bibliography.bib`. Glosarul de termeni și lista de acronime sunt gestionate prin pachetul `glossaries-extra`, cu intrările declarate în `5-glosar.tex`.

Lucrarea este versionată în Git într-un [repository](#) dedicat (`kookio-thesis`), separat de [repository](#)-ul codului sursă al aplicației *Kookio*, pentru a păstra istoricul de revizii al textului independent de cel al implementării.

A.2 Automatizarea compilării și reproductibilitatea

Compilarea lucrării este orchestrată de două scripturi [TypeScript](#) rulate prin `tsx` (executor [TypeScript](#) fără pas de transpilare separat), care partajează utilitățile comune din `utils.ts` (logger colorat, detectare de unelte, executare de procese-copil).

Scriptul `build.ts` orchestrează lanțul complet de compilare în ordinea `lualatex → makeglossaries → biber → lualatex → lualatex`. A treia rulare `lualatex` este o măsură defensivă: după ce a doua rulare expandează glosarul sau bibliografia, paginile se deplasează, iar referințele încrucișate din fișierul `.aux` proaspăt scris devin neactualizate; fără această rulare suplimentară, etichetele precum „Anexa B” pot apărea drept „Anexa ??” în PDF-ul final.

Scriptul `install-tex-deps.ts` citește fișierul de cerințe `tex-requirements.txt` (versionat în repository) și instalează pachetele lipsă prin `tlmgr`, optimizând operațiunea printr-un singur apel `tlmgr list -only-installed` folosit pentru deduplicare în memorie. Cele trei comenzi suportate sunt prezentate în Lista 1.

Lista 1: Comenzi principale de compilare și gestionare a dependențelor.

```
tsx build.ts ro                # compilare completă (cu bibliografie)
tsx build.ts ro --draft        # compilare rapidă (fără bibliografie)
tsx build.ts ro --delete       # curăță doar fișierele auxiliare

tsx install-tex-deps.ts        # instalează pachetele lipsă
tsx install-tex-deps.ts --check # verifică, fără modificări
tsx install-tex-deps.ts --update # actualizează tlmgr și pachetele
```

A.3 Compatibilitate multiplatformă

Configurațiile recomandate per platformă sunt următoarele:

- **macOS:** `brew install -cask mactex` instalează distribuția MacTeX cu schema completă (include implicit toate pachetele din `tex-requirements.txt`); rularea ulterioară a `install-tex-deps.ts` este opțională și servește la verificarea integrității.
- **Ubuntu / Debian:** se instalează schema de bază prin `sudo apt install texlive-base`, după care `install-tex-deps.ts` completează lista de pachete necesare. Alternativ, `texlive-full` acoperă toate dependențele într-un singur pas.
- **Windows:** se instalează MikTeX de la <https://miktex.org/download>; distribuția MikTeX detectează și instalează automat pachetele lipsă la prima compilare, motiv pentru care scriptul `install-tex-deps.ts` nu este necesar pe această platformă.

Fontul [FreeMono](#), folosit în mediul [TreeVerb](#) pentru caracterele [Unicode](#) de desenare a che- narelor (*box-drawing*, intervalul U+2500–U+257F), este o componentă GNU FreeFont care se instalează la nivelul sistemului de operare prin `brew install -cask font-gnu-freefont-freemono` pe macOS sau `sudo apt install fonts-freefont-ttf` pe Ubuntu/Debian.

Notă privind mediul TreeVerb. Mediul [TreeVerb](#) (definit local în preambulul lucrării, bazat pe `fvextra` și `fancyvrb`) utilizează fontul [FreeMono](#) pentru redarea structurii de fișiere în Secțiunea [3.1.1](#).

Anexa B Manifest Arhitectural PAN

Această anexă consolidează arhitectura PAN propusă în lucrare, sintetizând definițiile, justificările tehnologice și limitările identificate pe parcurs. Spre deosebire de Secțiunea 2 (care narează drumul către PAN) și Secțiunea 3 (care detaliază implementarea concretă în *Kookio*), prezenta anexă oferă o privire de ansamblu organizată tematic — utilă fie pentru o lectură de referință după ce lucrarea a fost parcursă integral, fie ca punct de plecare pentru cititorul care vrea să înțeleagă PAN fără să citească întregul studiu de caz.

B.1 Definiție și principii structurale

PAN (*Prisma–Analog–Nx*) este o arhitectură *full-stack* monolitică modulară pentru aplicații web construite integral în ecosistemul TypeScript. Cele trei componente eponime — Prisma ca ORM declarativ, Analog ca *meta-framework* Angular cu suport SSR și Nx ca orchestrator de *monorepo* — sunt elementele constitutive în jurul cărora se organizează o stivă coerentă cu cinci principii fundamentale:

1. **Limbaaj unic de la capăt la capăt.** Întreaga aplicație — UI, server, validare, persistență — este scrisă în TypeScript. Această decizie elimină categoria de erori provenite din desincronizarea contractelor între limbaje diferite (de exemplu, Java pe server și TypeScript pe client) și permite reutilizarea uniformă a tipurilor.
2. **O singură sursă de adevăr pentru contractele de date.** Schema bazei de date declarată în `schema.prisma` este sursa primară din care se derivă, automat, atât clientul Prisma TypeScript, cât și schemele de validare Zod (via `prisma-zod-generator`). Tipurile aplicaționale sunt apoi compuse din aceste scheme prin operatorii `.pick()`, `.omit()`, `.extend()` și `z.infer<>`. O modificare în `schema.prisma` se propagă, la compilare, prin toate straturile.
3. **Topologie modulară aciclică, aplicată la CI.** Codul este organizat într-un *monorepo* Nx cu cinci grupuri de biblioteci (`core`, `feature`, `prisma`, `server`, `shared`) și o relație de dependență strict aciclică. Regula `enforce-module-boundaries` respinge la integrarea continuă orice import care încalcă topologia, garantând că structura arhitecturală nu se degradează pe parcursul evoluției proiectului.
4. **Absența unui strat de repository intermediar.** Procedurile *tRPC* apelează direct clientul Prisma (`ctx.prisma`), fără un strat intermediar de tip *repository* orientat-obiect. Această decizie reduce numărul de straturi de adaptare și forțează ca logica de afaceri să se exprime în termenii primitivelor expuse de Prisma — un compromis explicit între simplitate și abstracție.

5. **Randare pe server ca mod implicit.** Combinația [Angular](#) + [Analog](#) livrează [SSR](#) drept comportament implicit pentru toate rutele, iar [SSG](#) este folosit punctual pentru paginile cu conținut stabil. Astfel, prima impresie a utilizatorului depinde de un [HTML](#) populat încă de la prima cerere, important atât pentru indexarea [SEO](#), cât și pentru perceperea performanței.

B.2 Stiva tehnologică detaliată

Fiecare nivel al stivei [PAN](#) este materializat de o tehnologie concretă, aleasă în raport cu alternative consacrate. Cele paisprezece niveluri sunt grupate în două tabele după responsabilitate: Tabelul 2 acoperă componentele aplicative ([frontend](#), server, persistență), iar Tabelul 3 acoperă tooling-ul de dezvoltare și operare.

Tabelul 2: Componentele aplicative ale arhitecturii [PAN](#) și justificarea fiecărei alegeri tehnologice.

Nivel	Tehnologie	Justificare
Frontend	Angular 20	Tipizare strictă nativă, ecosistem Material/-CDK consistent, semnale reactive
Meta-framework	Analog	Singura opțiune nativ- Angular cu SSR , rutare pe fișiere și <i>endpoint</i> -uri server integrate
Server runtime	Nitro	Adoptat de Analog , agnostic față de hosting (Vercel , Node.js, Bun, Deno)
API	tRPC	Tipuri partajate de la capăt la capăt fără un strat suplimentar de definiții
Validare	Zod via prisma-zod-generator	Generare automată din schema Prisma , integrare nativă cu tRPC
ORM	Prisma	Schemă declarativă, client TypeScript generat, migrații versionate
Bază de date	CockroachDB	Compatibilitate de protocol cu PostgreSQL , scalare orizontală nativă, nivel <i>serverless</i>
Cache	Redis	Suport nativ pentru <i>rate-limiting</i> , structuri de date specializate

Tabelul 3: Tooling-ul de dezvoltare și operare al arhitecturii PAN și justificarea fiecărei alegeri tehnologice.

Nivel	Tehnologie	Justificare
Monorepo	Nx	Filiație Angular (Nrwl), cache distribuit, generatoare native, <code>enforce-module-boundaries</code>
Build	Vite	Suport TypeScript nativ, <i>hot module replacement</i> rapid, compatibilitate directă cu Vitest
Test unitar	Vitest	Rulează pe Vite (nu duplică configurația), API compatibil Jest
Test E2E	Playwright	Suport multi-browser nativ, paralelism prin <i>workers</i> , depanare cu UI dedicată
CI/CD	GitHub Actions cu <code>composite actions</code>	Aliniere cu țintele Nx, reutilizare prin acțiuni compozite
Hosting	Vercel	Suport nativ pentru Analog/Nitro, mediu de previzualizare per ramură, variabile de mediu centralizate

B.3 Comparație cu arhitecturi alternative

PAN se înscrie într-o linie de arhitecturi `full-stack` moștenite conceptual din MEAN, propusă de Karpov în 2013 [19]. Figura 1 ilustrează paralela structurală între cele două.

Evoluția arhitecturală: MEAN → PAN

Paralelă structurală între stiva JavaScript tradițională (2013) și arhitectura propusă (2026)



OBSERVAȚII

- **Analog** absoarbe rolurile pe care MEAN le distribuia între Angular și Express, reflectând compresia arhitecturală a meta-cadrelor moderne.
- **Nx** adresează o preocupare (gestionarea monorepo cu cache distribuit) care nu exista ca subiect de discuție în era MEAN.
- **Prisma + CockroachDB** înlocuiesc MongoDB cu o stivă relațională ce păstrează flexibilitatea documentală prin tipul JSONB cu indexare GIN.

Figura 1: Paralelă structurală între arhitectura **MEAN** (Karpov, 2013) și arhitectura **PAN** propusă în prezenta lucrare. Diagrama evidențiază maturizarea ecosistemului: **Analog** absoarbe rolurile distribuite în **MEAN** între *Express* și *Angular*, iar **Nx** adresează preocuparea organizării **monorepo** care, în era **MEAN**, nu era încă subiect de discuție arhitecturală.

Spre deosebire de **MEAN**, care era un set de tehnologii relativ disjuncte unite prin convenție comunitară, **PAN** reflectă o filiație tehnologică directă: comunitatea Nrwl (**Nx**) este fondată de foști ingineri Google din echipa *Angular*, iar comunitatea **Analog** menține **Nx** ca platforma preferată de orchestrare. Această convergență structurală reduce frecușul de integrare pe care **MEAN** îl avea (fiecare dezvoltator **MEAN** își configura propriul flux de *build*).

Stiva contemporană React (MERN + **Next.js** + *Turborepo*) ocupă același spațiu de soluții ca **PAN**, dar într-un alt ecosistem. Tabelul 4 oferă o comparație directă.

Tabelul 4: Comparație între [MEAN](#) (2013), stiva React modernă și arhitectura [PAN](#).

Aspect	MEAN (2013)	MERN + Next.js + Turborepo	PAN
Frontend	Angular (vechi)	React	Angular 20
Meta-framework	— (Express)	Next.js	Analog
Server runtime	Node.js + Express	Node.js + Next.js	Nitro
API	REST manual	API routes + tRPC opțional	tRPC obligatoriu
Tipuri partajate	Nu	Parțial (App Router)	End-to-end automat
Bază de date	MongoDB	MongoDB sau PostgreSQL	CockroachDB
ORM	Mongoose	Prisma sau Drizzle	Prisma
Monorepo	—	Turborepo	Nx
Limbaj	JavaScript	TypeScript parțial	TypeScript pur

Diferențele majore între [PAN](#) și stiva React modernă sunt:

- **Comunitate.** [PAN](#) este construit pe [Angular](#), framework cu o cotă de utilizare mai mică decât a React (50,1 % față de 81,1 % *State of JavaScript 2024* [5]). Această diferență este atât o limitare practică (mai puțini dezvoltatori disponibili, mai puține biblioteci de [UI third-party](#)), cât și o invitație la disciplină — [Angular](#) impune o organizare mai structurată implicit, prin *dependency injection* și module.
- **Maturitate.** [Next.js](#) are aproape un deceniu de evoluție și o adopție extinsă în producție; [Analog](#) are sub trei ani și o comunitate mică. [PAN](#) compensează prin alegerea [Angular](#) și [Nitro](#) (ambele mature individual) ca platforme de bază, dar acceptă riscul comunitar al [Analog](#).
- **Tipizare end-to-end.** [tRPC](#) este în [PAN](#) o componentă obligatorie a arhitecturii, nu opțională. Această decizie face [PAN](#) mai restrictiv decât stiva React modernă (unde poți alege REST, GraphQL sau [tRPC](#) ad-hoc), dar elimină categoria de erori de desincronizare contractuală între client și server.

B.4 Limitări și compromisuri

Adoptarea [PAN](#) ca arhitectură nu este lipsită de compromisuri. Această subsecțiune le listează explicit, pentru ca un cititor care evaluează [PAN](#) pentru un proiect propriu să poată face o alegere informată:

- **Curba de adoptare.** [PAN](#) cere cunoștințe consolidate de [Angular](#) și de [TypeScript](#) avansat (*generics, conditional types, mapped types*). Pentru o echipă fără experiență [Angular](#), învățarea [PAN](#) are un cost semnificativ mai mare decât adoptarea unui [Next.js](#) convențional.
- **Maturitate [Analog](#).** [Analog](#) este, la momentul scrierii, un proiect cu o comunitate de câteva mii de utilizatori și fără adopție majoră în mediul corporativ. Riscul ca anumite probleme de producție să nu aibă răspuns documentat este real. Mitigarea folosită în prezenta lucrare a fost selectarea componentelor mature individual ([Angular](#), [Nitro](#)) și acceptarea explicită a riscului pentru stratul de orchestrare [Analog](#).
- **Cuplaj la [TypeScript](#).** [tRPC](#) presupune că ambele capete ale comunicării sunt [TypeScript](#). [PAN](#) nu este o opțiune viabilă pentru aplicații care trebuie să expună un [API](#) public consumat de clienți în alte limbaje (Java, Go, Python). În astfel de cazuri, GraphQL sau OpenAPI rămân alegeri mai potrivite.
- **Dependență moderată de furnizor.** Deși [PAN](#) poate fi publicat pe orice infrastructură compatibilă Node.js ([Nitro](#) suportă [Vercel](#), Netlify, AWS, Cloudflare, Bun, Deno), integrarea cu [Vercel](#) este cea mai bine bățătorită. Migrarea către un alt furnizor este posibilă, dar nu este lipsită de efort.
- **[CockroachDB](#) la scară mică.** Pentru aplicații sub 100 000 cereri pe zi, complexitatea operațională a [CockroachDB](#) (algoritmul de consens Raft, izolarea *serializable* implicită care necesită logică de reluare) este disproporționată față de [PostgreSQL](#) standard. [PAN](#) cu [PostgreSQL](#) clasic ar fi o variantă mai simplă pentru aplicații care nu necesită scalare orizontală reală.
- **Resurse de învățare limitate.** Conținutul comunitar (Stack Overflow, articole de blog, cursuri) pentru [PAN](#) ca atare nu există — există doar pentru componentele individuale. Un dezvoltator care întâmpină o problemă specifică integrării [Analog](#) + [tRPC](#) + [Prisma](#) + [Nx](#) trebuie să își construiască soluția pornind de la primele principii.

B.5 Direcții de evoluție arhitecturală

[PAN](#), ca arhitectură, este extensibilă pe mai multe axe care nu au fost explorate în studiul de caz *Kookio*:

- **Comunicare în timp real.** [tRPC](#) suportă *subscriptions* prin WebSocket sau Server-Sent Events; o ramură naturală de evoluție este integrarea cu un broker de mesaje ([Redis](#) Pub/Sub, NATS) pentru notificări către clienți autentificați.

- **Funcții la nivelul rețelei de margine (*edge functions*).** [Nitro](#) permite publicare parțială la nivelul *edge* ([Vercel](#) Edge Functions, [Cloudflare Workers](#)). Procedurile [tRPC](#) care nu necesită [Prisma](#) pot fi mutate la *edge* pentru reducerea latenței geografice.
- **Multi-regional.** [CockroachDB](#) suportă nativ replicare cross-zone; pentru o aplicație care iese din regimul mono-regional, această capacitate poate fi activată fără modificări de cod aplicativ.
- **Aplicații mobile.** Stiva [Angular](#) are suport pentru aplicații mobile native prin Capacitor sau NativeScript; bibliotecile core și shared ale unui proiect [PAN](#) sunt direct reutilizabile între o aplicație [web](#) și una mobilă.
- **Internaționalizare.** [Angular](#) i18n combinat cu rutare multilingvă [Analog](#) (rute parametrizate prin prefix de limbă) acoperă cazul aplicațiilor multilingve fără modificări arhitecturale.

[PAN](#) nu este, în concluzie, o arhitectură universal aplicabilă. Este o arhitectură pentru echipe mici cu cunoștințe [Angular](#) consolidate care construiesc aplicații [TypeScript](#) moderne, monolitice (în sensul publicării unitare) și centrate pe date. În acest spațiu de soluții, [PAN](#) oferă o coerență tehnologică și o disciplină structurală pe care alternativele bazate pe React le ating doar prin alegeri și convenții *ad-hoc*.

Glosar de termeni

Analog meta-framework full-stack pentru [Angular](#), cu suport [SSR](#) și rutare pe fișiere; folosit pentru aplicații [web](#)

Angular platformă și framework pentru aplicații [web](#) (SPA/[SSR](#))

Nitro runtime universal (UnJS) pentru rute server și randare pe server ([SSR](#))

Nx sistem de build și orchestrare pentru monorepo-uri JavaScript/TypeScript

Prisma bibliotecă [ORM](#) pentru TypeScript/Node.js

Zod bibliotecă de validare pe bază de schemă pentru TypeScript/JavaScript

avg medie aritmetică a unui set de valori; în SQL corespunde funcției AVG, care (în implementările uzuale) ignoră valorile NULL. Se folosește în raportare împreună cu percentile precum [p50](#) (mediană) și [p95](#)

backend stratul server-side care implementează logica de business, expune [API](#)-uri și mediază accesul la date (ex.: [Prisma](#) către [CockroachDB](#)). Poate rula ca [SSR](#), funcții [serverless/Edge Functions](#) sau servicii dedicate. În acest proiect, backend-ul este integrat în [Analog](#) (Nitro routes și routere [tRPC](#)).

biblatex pachet $\text{\LaTeX}$ pentru gestionarea bibliografiilor și citărilor cu backend-uri moderne (de ex. [biber](#))

CockroachDB sistem de gestiune a bazelor de date distribuit (NewSQL), compatibil la nivel de protocol cu [PostgreSQL](#), cu replicare și partiționare automate pentru reziliență și scalare orizontală

composite action tip de acțiune reutilizabilă în [GitHub](#) Actions care grupează mai mulți pași (run/uses) într-o singură acțiune; se distribuie ca director cu fișier `action.yml`, acceptă *inputs*, *outputs* și *secrets* și poate fi apelată dintr-un [workflow](#) prin `uses:..`. Utilă pentru a elimina duplicarea (DRY) în pipeline-urile [CI/CD](#); limitări uzuale: nu rulează în containere proprii și nu pot defini *services*.

darwin-arm64 etichetă de platformă (target) utilizată la generarea binary-uri pentru ecosistemul Node.js pe macOS (kernel *Darwin*) cu arhitectură ARM64 (Apple Silicon). În [Prisma](#), selectarea acestui target indică faptul că *engines* native sunt construite pentru dezvoltare locală pe macOS ARM și nu sunt compatibile cu Linux de producție; diferențele de CPU și biblioteci de sistem pot produce erori la runtime.

Edge Functions funcții executate în rețeaua de *edge* (aproape de utilizator), utile pentru latență redusă, personalizare și pre-/post-procesare a răspunsurilor [SSR](#); pot rula cu constrângeri mai stricte (timp CPU, memorie) față de funcțiile serverless clasice

framework set de convenții, componente și unelte care oferă o structură impusă pentru dezvoltarea de aplicații. Un *framework* controlează fluxul principal al aplicației (inversiune a controlului), definește puncte de extensie și impune un mod coerent de organizare a codului; spre deosebire de o bibliotecă, pe care aplicația o apelează explicit. Exemple în lucrare: [Angular](#); [Analog](#) funcționează ca *meta-framework* peste [Angular](#) pentru [SSR](#) și rute server.

FreeMono font monospațiat din familia GNU FreeFont, cu suport Unicode extins (inclusiv caractere *box-drawing* U+2500–U+257F)

frontend stratul aplicației care rulează în browser și gestionează [UI](#)-ul, logica de interacțiune și actualizările în [CSR](#). În arhitecturi hibride consumă HTML-ul generat prin [SSR/SSG](#) și trece prin [hidratare \(hydration\)](#). Comunică cu [backend](#)-ul prin [API](#) (ex.: [tRPC](#)). În acest proiect, frontend-ul este implementat cu [Angular](#) via [Analog](#).

full-stack abordare în care aceiași dezvoltatori/proiect acoperă cap-coadă [frontend](#)-ul și [backend](#)-ul, inclusiv modelarea datelor și pipeline-urile de [CI/CD](#). Beneficii: tipuri partajate, coerență arhitecturală și viteză de livrare. În această lucrare, modelul full-stack este realizat prin arhitectura [PAN](#) într-un [monorepo](#) Nx.

GitHub platformă de găzduire și colaborare pentru proiecte software bazate pe sistemul de versiuni Git; oferă găzduire de [repository-uri](#), mecanisme de colaborare (pull requests, code review), management de *issues* și [CI/CD](#) prin GitHub Actions; utilizată frecvent pentru automatizări și publicare.

GitLab platformă de găzduire și colaborare pentru proiecte software bazate pe Git, disponibilă ca serviciu cloud sau instalare self-hosted; integrează nativ [CI/CD](#) (GitLab CI/CD), management de *issues*, *merge requests* și registru de containere; alternativă la [GitHub](#) cu accent pe lanțul DevOps unificat.

hidratare (hydration) procesul prin care codul din client atașează comportament interactiv la *markup*-ul generat prin [SSR](#); permite trecerea de la randare inițială pe server la interacțiuni [CSR](#) fără reîncărcarea paginii

Kookio aplicație [web](#) full-stack folosită ca studiu de caz în această lucrare pentru gestiunea cămării, planificarea zilnică a meselor și derivarea automată a listei de cumpărături, construită pe arhitectura [PAN](#)

jurnalizare (logging) colectarea și înregistrarea evenimentelor aplicației (mesaje structurate pe niveluri, corelare prin identificatori, export către sisteme de agregare) pentru depanare și observabilitate

LuaLaTeX motor $\text{\LaTeX}$ bazat pe LuaTeX, cu suport nativ pentru Unicode și fonturi OpenType

meta-framework framework care extinde un framework de bază pentru capabilități full-stack (ex. rutare pe fișiere, [SSR/SSG](#), integrare cu [tRPC](#)). În această lucrare, [Analog](#) extinde [Angular](#) peste runtime-ul [Nitro](#).

monorepo strategie de organizare a codului în care mai multe aplicații și biblioteci coexistă într-un singur depozit, cu orchestrare unitară a build-urilor, testelor și [CI/CD](#); în contextul lucrării este gestionat cu Nx

Next.js framework React orientat pe randare hibridă ([SSR/SSG](#)), frecvent asociat cu JAMstack

Pino bibliotecă de logare pentru Node.js, orientată pe performanță (JSON logs, output low-overhead)

Playwright bibliotecă pentru automatizarea browserelor și testare [E2E](#), cu suport pentru Chromium, Firefox și WebKit, rulare headless/GUI și unelte precum trace viewer

PostgreSQL sistem de gestiune a bazelor de date relaționale open-source, cu suport extins pentru standardul [SQL](#), tranzacții ACID și extensibilitate prin tipuri și funcții definite de utilizator

Prisma Client client TypeScript pentru baza de date, generat automat de [Prisma](#) din fișierul `schema.prisma`; oferă un API tipizat pentru operații [SQL](#) (CRUD, filtre, relații, tranzacții) asupra modelelor definite. Utilizează motoare native (Query Engine) distribuite ca binary-uri dependente de platformă (ex.: [darwin-arm64](#), [rhel-openssl-3.0.x](#)), motiv pentru care se generează pe mediul țintă de deploy; se invocă prin `npx prisma generate`.

randare procesul prin care o aplicație generează HTML (și actualizări ale interfeței) din cod și date. În web-ul modern poate avea loc pe server ([SSR](#)), la build-time prin [SSG](#)/prerender, sau în browser ([CSR](#)); de regulă, este urmată de [hidratare \(hydration\)](#) pentru interactivitate. În [Analog](#), strategia de randare poate fi aleasă per rută.

Redis magazin de date *in-memory* tip cheie-valoare, cu structuri avansate (liste, seturi, hash-uri, mulțimi ordonate, *streams*) și latență foarte mică; folosit frecvent pentru caching, stocare sesiuni, cozi/mesagerie (Pub/Sub, Streams), rate-limiting și *leaderboards*. Suportă persistență opțională (RDB/AOF), replicare, *cluster* pentru partiționare și, de regulă, completează o bază [SQL/DB](#) principală în arhitecturi microservicii și [serverless/Edge Functions](#).

repository spațiu logic pentru stocarea codului sursă, a istoricului de versiuni și a metadatelor; poate fi local sau la distanță (remote, ex.: GitHub/GitLab). În această lucrare, monorepo-ul reunește mai multe proiecte într-un singur repository.

rhel-openssl-3.0.x etichetă de platformă pentru distribuții Linux compatibile Red Hat Enterprise Linux care folosesc biblioteca OpenSSL 3.0.x. În [Prisma](#), acest target este folosit în `binaryTargets` pentru a genera binary-uri compatibile cu mediile de execuție tipice de producție (ex.: containere bazate pe *glibc*); alegerea corectă elimină erorile de tip `wrong platform for query engine` la deploy.

serverless model operațional în care alocarea resurselor și scalarea sunt gestionate de platformă; dezvoltatorul furnizează funcții sau servicii ce pornesc la cerere, cu facturare pe utilizare; folosit adesea pentru [API-uri](#), [SSR](#) și joburi programate

TreeVerb mediu *verbatim* personalizat (bazat pe `fvextra/fancyvrb`) pentru randarea arborilor/structurilor de fișiere; folosește [FreeMono](#) pentru simboluri *box-drawing*

TypeScript superset tipizat al limbajului JavaScript care introduce tipuri statice și se transpilează la JavaScript

Unicode standard de codificare a caracterelor care oferă o mapare unică pentru simboluri folosite în scrieri și tehnică (ex. caractere *box-drawing*)

Vercel platformă cloud pentru build, preview și deploy de aplicații [web](#), cu integrare [CI/CD](#), variabile de mediu per mediu și suport pentru [SSR](#), funcții [serverless](#) și [Edge Functions](#)

Vite instrument de build și server de dezvoltare pentru aplicații [web](#) moderne, bazat pe module ECMAScript, cu pornire foarte rapidă și *hot module replacement* (HMR); stă la baza [Vitest](#) și este folosit de [Analog](#) pentru [SSR](#) și build

Vitest framework de testare rapid pentru ecosistemul Vite, orientat spre teste unitare și de integrare, cu API în mare parte compatibil Jest și suport nativ pentru TypeScript

web ecosistemul de resurse *hypertext* accesibile prin Internet (World Wide Web); în lucrare, „aplicație web” desemnează aplicație accesată din browser, cu randare [SSR/SSG/CSR](#)

workflow configurație declarativă care descrie o succesiune de pași automatizați (build, testare, lint, deploy) declanșați de evenimente (ex.: push, pull request, programare). În ecosisteme precum [GitHub Actions](#) și [GitLab CI](#), un workflow este compus din job-uri și pași, poate folosi matrici, cache, artefacte, secrete/variabile și reguli de concurență. În acest proiect, workflow-urile orchestrează [CI/CD](#) și aliniaza execuția la țintele [Nx](#).

Lista de acronime

API interfață de programare a aplicațiilor

CI integrare continuă

CI/CD integrare continuă și livrare/implementare continuă

CSR randare pe client

DB bază de date

DOM model al obiectelor documentului

DTO obiect de transfer de date

E2E testare cap-coadă

HTML HyperText Markup Language (limbaj de marcare hipertext)

HTTP protocol de transfer hipertext

MEAN MongoDB–Express–Angular–Node.js

MVP produs minim viabil

ORM mapare obiect–relațională

p50 percentila 50 (mediană)

p95 percentila 95

PAN Prisma–Analog–Nx

PR cerere de integrare (pull request)

SEO optimizare pentru motoarele de căutare

SPA aplicație web de o singură pagină (single-page application)

SQL limbaj de interogare structurat

SSG generare statică de pagini

SSR randare pe server

tRPC apeluri de proceduri la distanță tipizate pentru TypeScript

UI interfață de utilizator

URL localizator uniform de resurse

Referințe

- [1] Autorii Pino, *Pino — Logger pentru Node.js*, Documentația Pino, Documentație oficială Pino; citată pentru logarea centralizată utilizată atât în backend, cât și în frontend., 2025, URL: <https://getpino.io/#/docs/transport> (accesat în 18.5.2026).
- [2] Autorii Zod, *Documentația Zod*, Documentația Zod, Documentație oficială Zod; citată pentru validarea schemelor utilizate împreună cu tipurile generate de Prisma., 2025, URL: <https://zod.dev/?id=objects> (accesat în 18.5.2026).
- [3] Cockroach Labs, *Documentația CockroachDB*, Documentația CockroachDB, Documentație oficială CockroachDB; citată în planul de dezvoltare pentru analiza scalabilității orizontale., 2025, URL: <https://www.cockroachlabs.com/docs/stable/postgresql-compatibility> (accesat în 18.5.2026).
- [4] Omar Dulaimi și colaboratorii, *Generatorul Prisma Zod*, Depozitul GitHub, Versiunea 1.27.7; citată pentru generarea automată a schemelor de validare Zod pe baza modelelor Prisma., 2025, URL: <https://github.com/omar-dulaimi/prisma-zod-generator> (accesat în 18.5.2026).
- [5] Devographics, *State of JavaScript 2024*, Sondaj anual al ecosistemului JavaScript, Sondaj anual al comunității JavaScript (peste 22.000 de respondenți); citat pentru tiparele de utilizare a SSR (59%) și SSG (46%), adopția TypeScript și complexitatea framework-urilor frontend., 2024, URL: <https://2024.stateofjs.com/en-US/libraries/frontend-frameworks/> (accesat în 18.5.2026).
- [6] Dezvoltatorii AnalogJS, *Analog — Meta-framework full-stack pentru Angular*, Documentația Analog, Documentație oficială Analog; citată pentru SSR, rutare bazată pe fișiere și integrarea Nitro/tRPC în aplicațiile Angular., 2025, URL: <https://analogjs.org/docs/features/server-side-rendering> (accesat în 18.5.2026).
- [7] Dezvoltatorii AnalogJS, *Routing — Rutare bazată pe structura de fișiere*, Documentația Analog, Documentație oficială Analog; citată pentru rutarea *filesystem-based* (*.page.ts, rute dinamice cu parametri, colocarea logicii de server în *.server.ts)., 2025, URL: <https://analogjs.org/docs/features/routing/overview> (accesat în 18.5.2026).
- [8] Echipa Google Angular, *Angular Material — Componente cu accesibilitate nativă*, Documentația Angular Material/CDK, Documentație oficială Angular Material; citată pentru setul de componente UI și pentru Angular CDK, consistent în privința accesibilității (*a11y*)., 2025, URL: <https://material.angular.dev/> (accesat în 18.5.2026).
- [9] Echipa Google Angular, *Prezentare generală a semnalelor (Signals Overview)*, Documentația Angular, Documentație oficială Angular; citată pentru modelul de reactivitate (Signals) utilizat în componente independente și în contextul randării pe server (SSR)., 2025, URL: <https://angular.dev/guide/signals> (accesat în 15.6.2025).
- [10] Echipa Microsoft Playwright, *Testarea cu Playwright — Ghid introductiv*, Documentația Playwright, Documentație oficială Playwright; citată pentru testarea end-to-end adoptată în perioada iunie–iulie 2025., 2025, URL: <https://playwright.dev/docs/test-global-setup-teardown> (accesat în 18.5.2026).
- [11] Echipa Nrwl, *Ce este Nx?*, Documentația Nx, Documentație oficială Nx; citată pentru orchestrarea monorepo-ului, rulările afectate, caching și graful de proiect., 2025, URL: <https://nx.dev/docs/features/ci-features/affected> (accesat în 18.5.2026).

- [12] Echipa Nrwl, *Enforce Module Boundaries — Regulă ESLint pentru constrângeri arhitecturale*, Documentația Nx, Documentație oficială Nx; citată pentru regula `@nx/enforce-module-boundaries` care respinge la CI importurile care încalcă topologia dintre grupuri de biblioteci., 2025, URL: <https://nx.dev/docs/features/enforce-module-boundaries> (accesat în 18.5.2026).
- [13] Echipa UnJS, *Nitro: Motorul de server pentru web*, Documentația Nitro, Documentație oficială Nitro; citată pentru execuția pe server, rutele API și mediile de implementare compatibile cu Analog SSR., 2025, URL: <https://nitro.build/guide> (accesat în 18.5.2026).
- [14] Echipa Vite, *Ghidul Vite*, Documentația Vite, Documentație oficială Vite; citată pentru configurarea build-ului comună aplicațiilor și bibliotecilor și integrarea cu țintele Nx., 2025, URL: <https://vitejs.dev/guide/why.html> (accesat în 18.5.2026).
- [15] Echipa Vitest, *Ghidul Vitest*, Documentația Vitest, Documentație oficială Vitest; citată pentru configurarea și utilizarea testelor unitare, folosită în proiect începând cu august 2025., 2025, URL: <https://vitest.dev/guide/workspace.html> (accesat în 18.5.2026).
- [16] Eurostat, *Food waste: 132 kg per inhabitant in the EU in 2022*, Comunicat Eurostat, Comunicat oficial Eurostat; citat pentru date privind risipa alimentară per locuitor în Uniunea Europeană (132 kg/locuitor în 2022, 59,2 milioane tone total) și pentru ponderea gospodăriilor (54%) în risipa totală., 2024, URL: <https://ec.europa.eu/eurostat/web/products-eurostat-news/w/ddn-20240927-2> (accesat în 17.5.2026).
- [17] Grupul Global de Dezvoltare PostgreSQL, *Documentația oficială PostgreSQL*, Documentația PostgreSQL, Documentație oficială PostgreSQL; citată pentru protocolul de legătură (*wire protocol*) pe care **CockroachDB** îl emulează pentru compatibilitate cu ecosistemul de unelte SQL., 2025, URL: <https://www.postgresql.org/docs/current/protocol.html> (accesat în 18.5.2026).
- [18] JetBrains, *JavaScript and TypeScript Trends 2024*, Insights From the Developer Ecosystem Survey, Sondaj anual JetBrains; citat pentru creșterea adopției TypeScript (12% în 2017, 35% în 2024) și ponderea dezvoltatorilor predominant TypeScript (67%)., 2024, URL: <https://blog.jetbrains.com/webstorm/2024/02/js-and-ts-trends-2024/> (accesat în 17.5.2026).
- [19] Valeri Karpov, *The MEAN Stack: MongoDB, ExpressJS, AngularJS and Node.js*, MongoDB Blog, Articol original prin care Valeri Karpov a introdus termenul „MEAN stack”; citat ca referință istorică pentru arhitectura tradițională din ecosistemul JavaScript și pentru paralela conceptuală cu arhitectura PAN propusă în prezenta lucrare., Apr. 2013, URL: <http://blog.mongodb.org/post/49262866911/the-mean-stack-mongodb-expressjs-angularjs-and> (accesat în 17.5.2026).
- [20] Parlamentul European, *Food waste in Europe: facts, EU policies and 2030 targets*, European Parliament Topics, Sinteza oficială a Parlamentului European; citată pentru cauzele comportamentale ale risipei alimentare la nivel de consumator (cumpărături impulsive, planificare insuficientă a meselor, gestionare deficitară a inventarului) și pentru obiectivele 2030., 2025, URL: <https://www.europarl.europa.eu/topics/en/article/20240318ST019401/food-waste-in-europe-facts-eu-policies-and-2030-targets> (accesat în 17.5.2026).